Numeric Functions — Rounding, Math, and Type Casting

> ***Chapter 16 | PostgreSQL Complete Guide*** *Module: 02_SQL_Basics | Difficulty: Beginner–Intermediate | Estimated Reading Time: 35 minutes*

Table of Contents

Learning Objectives

By the end of this chapter, you will be able to:

- Apply ROUND, CEIL, FLOOR, and TRUNC to control decimal precision and direction
- Use ABS, MOD, POWER, and SQRT for mathematical computations in SQL
- Generate random numbers and random samples using RANDOM() and TABLESAMPLE
- Understand GREATEST and LEAST for row-level min/max across columns
- Perform safe numeric type casting and understand overflow/precision behavior

---

Theory: Numeric Types in PostgreSQL

PostgreSQL provides three families of numeric types:

Type	Storage	Range	Notes
`SMALLINT`	2 bytes	-32,768 to 32,767	

`INTEGER` / `INT`	4 bytes	-2.1B to 2.1B	Most common integer
`BIGINT`	8 bytes	$\pm 9.2 \times 10^{18}$	For large counts, IDs
`NUMERIC(p,s)`	variable	arbitrary precision	Exact; for money, accounting
`DECIMAL(p,s)`	variable	same as NUMERIC	Alias for NUMERIC
`REAL`	4 bytes	6 decimal digits precision	Floating-point (inexact)
`DOUBLE PRECISION`	8 bytes	15 decimal digits precision	Floating-point (inexact)
`SERIAL` / `BIGSERIAL`	4/8 bytes	auto-increment	NEXTVAL wrapper

Exact vs Inexact Arithmetic

`NUMERIC` / `DECIMAL` is **exact** — safe for financial calculations. `REAL` / `DOUBLE PRECISION` is **inexact** — subject to floating-point rounding errors.

```
SELECT 0.1 + 0.2;                 -- 0.30000000000000004 (double precision!)
SELECT 0.1::NUMERIC + 0.2::NUMERIC; -- 0.3 (exact)
```

Rounding — ROUND, CEIL, FLOOR, TRUNC

ROUND

```
ROUND(numeric)           -- round to nearest integer
ROUND(numeric, scale)    -- round to scale decimal places
```

Rounding rule: .5 rounds away from zero (standard "half-up").

```
SELECT ROUND(4.5);          -- 5
SELECT ROUND(4.4);          -- 4
SELECT ROUND(-4.5);         -- -5  (away from zero)
SELECT ROUND(3.14159, 2);   -- 3.14
SELECT ROUND(3.14159, 4);   -- 3.1416
SELECT ROUND(1234.5, -2);   -- 1200 (negative scale rounds to hundreds)
SELECT ROUND(1234.5, -3);   -- 1000 (round to thousands)
```

CEIL / CEILING

Always rounds **up** toward positive infinity:

```
SELECT CEIL(4.1);    -- 5
SELECT CEIL(4.9);    -- 5
SELECT CEIL(-4.1);   -- -4  (up toward positive infinity)
SELECT CEIL(-4.9);   -- -4
SELECT CEILING(4.5); -- 5  (CEILING is alias for CEIL)
```

FLOOR

Always rounds **down** toward negative infinity:

```
SELECT FLOOR(4.9);   -- 4
SELECT FLOOR(4.1);   -- 4
SELECT FLOOR(-4.1);  -- -5  (down toward negative infinity)
SELECT FLOOR(-4.9);  -- -5
```

TRUNC

Truncates toward zero (drops fractional part, no rounding):

```
SELECT TRUNC(4.9);      -- 4   (truncate, don't round up)
SELECT TRUNC(-4.9);     -- -4  (toward zero, not negative infinity)
SELECT TRUNC(3.14159, 2); -- 3.14 (truncate to 2 decimal places)
SELECT TRUNC(1234.5, -2); -- 1200 (truncate to hundreds)
```

Comparison: ROUND vs CEIL vs FLOOR vs TRUNC

```
Value:    4.1    4.5    4.9   -4.1   -4.5   -4.9
ROUND:      4      5      5     -4     -5     -5
CEIL:       5      5      5     -4     -4     -4
FLOOR:      4      4      4     -5     -5     -5
TRUNC:      4      4      4     -4     -4     -4
```

Absolute Value and Sign — ABS, SIGN

```
ABS(numeric)    -- absolute value (distance from zero)
SIGN(numeric)   -- returns -1, 0, or 1
```

```
SELECT ABS(-42);      -- 42
SELECT ABS(42);       -- 42
SELECT ABS(0);        -- 0
SELECT ABS(-3.14);    -- 3.14

SELECT SIGN(-42);     -- -1
SELECT SIGN(0);       --  0
SELECT SIGN(42);      --  1
SELECT SIGN(-0.001);  -- -1

-- Practical: find rows where change is significant in either direction
SELECT * FROM price_changes WHERE ABS(pct_change) > 10;
```

Modulo and Integer Division — MOD, DIV

MOD (Modulo / Remainder)

```
MOD(dividend, divisor)    -- function form
dividend % divisor         -- operator form
```

```
SELECT MOD(10, 3);    -- 1   (10 = 3*3 + 1)
SELECT 10 % 3;        -- 1
SELECT MOD(10, 2);    -- 0   (even number)
SELECT MOD(7, 4);     -- 3
SELECT MOD(-7, 4);    -- -3  (sign follows dividend in PostgreSQL)
```

DIV — Integer Division

```
SELECT DIV(10, 3);    -- 3   (floor division)
SELECT 10 / 3;        -- 3   (integer division when both operands are integers)
SELECT 10.0 / 3;      -- 3.3333...  (float division when operand is decimal)
```

Practical Applications

```
-- Check if a number is even/odd
SELECT employee_id, MOD(employee_id, 2) = 0 AS is_even FROM employees;

-- Convert seconds to hours, minutes, seconds
SELECT total_seconds,
       DIV(total_seconds, 3600) AS hours,
       DIV(MOD(total_seconds, 3600), 60) AS minutes,
       MOD(total_seconds, 60) AS seconds
FROM   timer_logs;
```

Power and Roots — POWER, SQRT, CBRT

```
POWER(base, exponent)     -- base ^ exponent
base ^ exponent           -- operator form (PostgreSQL-specific)
SQRT(number)              -- square root
CBRT(number)              -- cube root (PostgreSQL-specific)
```

```
SELECT POWER(2, 10);     -- 1024
SELECT 2 ^ 10;           -- 1024
SELECT POWER(2, 0.5);    -- 1.4142135... (square root via power)
SELECT SQRT(16);         -- 4
SELECT SQRT(2);          -- 1.4142135623730951
SELECT CBRT(27);         -- 3  (cube root: 3*3*3 = 27)
SELECT CBRT(8);          -- 2

-- Euclidean distance
```

```sql
SELECT SQRT(POWER(x2 - x1, 2) + POWER(y2 - y1, 2)) AS distance
FROM   coordinates;
```

Random Numbers — RANDOM, SETSEED

RANDOM()

Returns a double precision value in `[0.0, 1.0)` — inclusive of 0, exclusive of 1.

```sql
SELECT RANDOM();                       -- e.g., 0.7182818...
SELECT FLOOR(RANDOM() * 100)::INT;     -- random int 0-99
SELECT FLOOR(RANDOM() * 100 + 1)::INT; -- random int 1-100

-- Random sample from a table (approximate)
SELECT * FROM large_table ORDER BY RANDOM() LIMIT 100;
-- NOTE: ORDER BY RANDOM() is slow on large tables; prefer TABLESAMPLE
```

SETSEED

Sets the seed for reproducible random sequences:

```sql
SELECT SETSEED(0.5);
SELECT RANDOM();  -- always the same value after this seed
SELECT RANDOM();  -- next value in the sequence
```

TABLESAMPLE (faster than ORDER BY RANDOM())

```sql
-- Bernoulli: each row independently included with given probability
SELECT * FROM customers TABLESAMPLE BERNOULLI(5);  -- ~5% of rows

-- System: sample by page (faster, less random)
SELECT * FROM customers TABLESAMPLE SYSTEM(1);     -- ~1% of pages
```

Extremes — GREATEST, LEAST

These operate across **columns within a single row** (not across rows like MAX/MIN aggregates):

```sql
GREATEST(val1, val2, ..., valN)  -- returns largest non-NULL value
LEAST(val1, val2, ..., valN)     -- returns smallest non-NULL value
```

```sql
SELECT GREATEST(1, 5, 3, 9, 2);   -- 9
SELECT LEAST(1, 5, 3, 9, 2);      -- 1

-- Row-level max of two date columns
SELECT order_id,
       GREATEST(promised_date, actual_ship_date) AS later_date
FROM   shipments;
```

```sql
-- Clamp a value within a range [min_val, max_val]
SELECT GREATEST(0, LEAST(100, score)) AS clamped_score FROM tests;

-- NULL behavior: GREATEST/LEAST return NULL if ANY argument is NULL
SELECT GREATEST(1, NULL, 3);   -- NULL
SELECT LEAST(NULL, 5, 2);      -- NULL
```

Logarithms and Trigonometry

```sql
-- Natural log (base e)
SELECT LN(2.71828);      -- ~1.0
SELECT LN(100);          -- 4.605...

-- Log base 10
SELECT LOG(100);         -- 2 (PostgreSQL: LOG(x) is log base 10)
SELECT LOG(10, 1000);    -- 3 (LOG(base, x))

-- Log base 2
SELECT LOG(2, 8);        -- 3

-- Exponential (e^x)
SELECT EXP(1);           -- 2.71828... (e)
SELECT EXP(2);           -- 7.38905...

-- Pi constant
SELECT PI();             -- 3.14159265358979

-- Trigonometric (arguments in radians)
SELECT SIN(PI() / 2);   -- 1.0
SELECT COS(0);           -- 1.0
SELECT TAN(PI() / 4);   -- ~1.0
SELECT ASIN(1);          -- PI()/2
SELECT DEGREES(PI());    -- 180
SELECT RADIANS(180);     -- PI()
```

Numeric Type Casting

Cast Operators

```sql
-- CAST function (SQL standard)
CAST(expression AS target_type)

-- :: operator (PostgreSQL-specific shorthand)
expression::target_type
```

```sql
SELECT CAST('42' AS INTEGER);         -- 42
SELECT '42'::INTEGER;                 -- 42
SELECT CAST(42.9 AS INTEGER);         -- 42 (truncates, does not round)
SELECT 42.9::INTEGER;                 -- 42

-- Integer to float
SELECT 10::DOUBLE PRECISION / 3;      -- 3.3333...
SELECT 10 / 3;                        -- 3 (integer division!)
SELECT 10.0 / 3;                      -- 3.3333...

-- Text to numeric
SELECT '3.14'::NUMERIC;               -- 3.14
SELECT '1,234.56'::NUMERIC;           -- ERROR: invalid input (commas not allowed)
SELECT REPLACE('1,234.56', ',', '')::NUMERIC;  -- 1234.56
```

Safe Casting with TRY/CATCH Pattern

```sql
-- PostgreSQL doesn't have TRY_CAST, but you can use a function or regex:
SELECT CASE
           WHEN value ~ '^-?[0-9]+(\.[0-9]+)?$'
           THEN value::NUMERIC
           ELSE NULL
       END AS safe_numeric
FROM   raw_data;
```

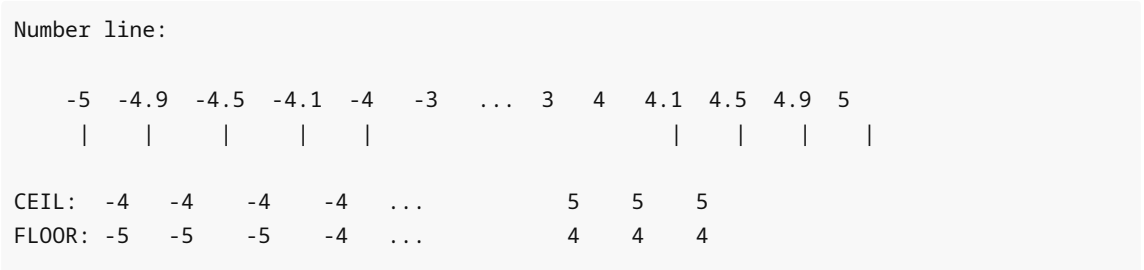
Numeric Precision

```sql
-- NUMERIC(precision, scale)
-- precision = total significant digits
-- scale = digits after decimal point
CAST(3.14159 AS NUMERIC(5, 2));    -- 3.14  (rounded to 2 decimal places)
CAST(3.14159 AS NUMERIC(10, 5));   -- 3.14159

-- Overflow:
CAST(123456 AS NUMERIC(5, 2));     -- ERROR: numeric field overflow
```

ASCII Visual Diagrams

Rounding Direction Comparison



```
ROUND: -5   -5    -4    -4   ...          4    5    5
TRUNC: -4   -4    -4    -4   ...          4    4    4

CEIL  = always towards +infinity (ceiling above you)
FLOOR = always towards -infinity (floor below you)
ROUND = towards nearest integer (away from 0 on .5)
TRUNC = always towards 0 (chop off fraction)
```

GREATEST/LEAST vs MAX/MIN

```
GREATEST/LEAST = across COLUMNS within one row
MAX/MIN        = across ROWS within one column

Row:                                Group:
+----+------+------+------+         +----------+--------+
| id | col1 | col2 | col3 |         | category | sales  |
+----+------+------+------+         +----------+--------+
| 1  |  10  |   7  |  15  |         | A        |    100 |
+----+------+------+------+         | A        |    200 |
                                    | A        |    150 |
GREATEST(col1,col2,col3) = 15       +----------+--------+
LEAST(col1,col2,col3)    = 7        MAX(sales) = 200 (across rows)
```

Integer vs Float Division

```
Expression      Type arithmetic    Result
----------      ---------------    ------
10 / 3          INT / INT      =   3      (integer, truncated)
10 / 3.0        INT / NUMERIC  =   3.333  (promoted to numeric)
10.0 / 3        NUMERIC / INT  =   3.333  (promoted to numeric)
10::FLOAT / 3   FLOAT / INT    =   3.333  (explicit cast)
```

SQL Examples

```sql
-- Example 1: ROUND to 2 decimal places
SELECT product_name, ROUND(unit_price, 2) AS price FROM products;

-- Example 2: ROUND to nearest 100
SELECT ROUND(1250, -2);   -- 1300

-- Example 3: CEIL for page count
SELECT CEIL(COUNT(*)::FLOAT / 10) AS total_pages FROM employees;

-- Example 4: FLOOR for age from fractional years
SELECT employee_id, FLOOR(EXTRACT(EPOCH FROM AGE(birth_date)) / 31557600) AS
age_years
FROM   employees;

-- Example 5: TRUNC for financial truncation (not rounding)
```

```sql
SELECT TRUNC(transaction_amount, 2) AS truncated_cents FROM transactions;

-- Example 6: ABS for absolute difference
SELECT product_a, product_b,
       ABS(price_a - price_b) AS price_difference
FROM   price_comparison;

-- Example 7: SIGN to categorize
SELECT employee_id, profit_loss,
       CASE SIGN(profit_loss)
           WHEN  1 THEN 'Profit'
           WHEN  0 THEN 'Break Even'
           WHEN -1 THEN 'Loss'
       END AS status
FROM   performance;

-- Example 8: MOD for even/odd
SELECT id, name, CASE WHEN MOD(id, 2) = 0 THEN 'even' ELSE 'odd' END AS parity
FROM   items;

-- Example 9: MOD for cyclic logic (e.g., day of week cycling)
SELECT MOD(day_number - 1 + 3, 7) + 1 AS shifted_day FROM schedule;

-- Example 10: Integer division for time conversion
SELECT duration_seconds,
       DIV(duration_seconds, 3600)                          AS hours,
       DIV(MOD(duration_seconds, 3600), 60)                 AS minutes,
       MOD(duration_seconds, 60)                            AS seconds
FROM   recordings;

-- Example 11: POWER for compound interest
SELECT principal,
       rate,
       years,
       principal * POWER(1 + rate, years) AS future_value
FROM   investments;

-- Example 12: SQRT for standard deviation check
SELECT SQRT(AVG((value - avg_value)^2)) AS std_dev
FROM   (SELECT value, AVG(value) OVER () AS avg_value FROM measurements) sub;

-- Example 13: RANDOM() for random sampling
SELECT * FROM customers ORDER BY RANDOM() LIMIT 5;

-- Example 14: RANDOM() for random int in range [1, 100]
SELECT FLOOR(RANDOM() * 100 + 1)::INT AS random_priority FROM tasks;

-- Example 15: GREATEST for row-level max
SELECT order_id,
       GREATEST(est_delivery, actual_delivery, promised_date) AS latest_date
FROM   shipments;
```

```sql
-- Example 16: LEAST for clamping
SELECT score,
       GREATEST(0, LEAST(100, score)) AS clamped_score
FROM   student_results;

-- Example 17: LOG for logarithmic scale
SELECT id, value, LOG(NULLIF(value, 0)) AS log_value FROM measurements;

-- Example 18: Type casting integer division
SELECT 7 / 2 AS integer_div,         -- 3
       7::FLOAT / 2 AS float_div,     -- 3.5
       7 / 2.0 AS mixed_div;          -- 3.5

-- Example 19: NUMERIC precision
SELECT CAST(total * 0.0825 AS NUMERIC(12, 2)) AS tax FROM invoices;

-- Example 20: ROUND for financial reporting
SELECT
    SUM(amount)                     AS total_raw,
    ROUND(SUM(amount), 2)           AS total_rounded,
    ROUND(AVG(amount), 4)           AS avg_rounded,
    CEIL(MAX(amount))               AS max_ceiling,
    FLOOR(MIN(amount))              AS min_floor
FROM transactions;
```

Common Mistakes

1. **Integer division when float result is expected.** `7 / 2` in SQL returns `3`, not `3.5`. Always cast at least one operand: `7::FLOAT / 2` or `7 / 2.0`.
2. **Using ROUND on FLOAT type.** `ROUND(0.1 + 0.2, 2)` on floating-point may still give `0.30000000000000004`. Use `NUMERIC` type for monetary values: `ROUND((0.1::NUMERIC + 0.2::NUMERIC), 2)`.
3. **Misunderstanding TRUNC vs FLOOR for negative numbers.** `TRUNC(-4.7)` = -4 (toward zero); `FLOOR(-4.7)` = -5 (toward negative infinity). Use TRUNC when you want to "chop off" the decimal; use FLOOR when you need the mathematical floor.
4. **GREATEST/LEAST returning NULL when any argument is NULL.** Unlike COALESCE, GREATEST/LEAST return NULL if any argument is NULL. Use `GREATEST(COALESCE(a,0), COALESCE(b,0))` to treat NULLs as zero.
5. **ORDER BY RANDOM() for large tables.** This generates a random value for every row and sorts them all — O(n log n). For large tables, use `TABLESAMPLE` instead.
6. **LOG vs LN confusion.** In PostgreSQL, `LOG(x)` is log base 10, not natural log. Use `LN(x)` for natural logarithm. `LOG(base, x)` is the two-argument form for arbitrary base.
7. **Casting text with commas or currency symbols to NUMERIC.** `'$1,234.56'::NUMERIC` raises an error. Strip formatting first: `REPLACE(REPLACE(amount, '$', ''), ',', '')::NUMERIC`.

Best Practices

1. **Use `NUMERIC` / `DECIMAL` for all monetary values** — floating-point imprecision is unacceptable for financial calculations. Store as `NUMERIC(15, 2)` for most currencies.
2. **Explicit casts make intent clear.** Write `10::FLOAT / 3` rather than `10.0 / 3` — the intent to perform floating-point division is obvious.
3. **Use `NULLIF` before division to prevent division-by-zero errors.** `numerator / NULLIF(denominator, 0)` returns NULL instead of raising an error.
4. **Clamp values with GREATEST/LEAST** rather than CASE WHEN for concise range enforcement: `GREATEST(min_val, LEAST(max_val, input))`.
5. **Avoid ORDER BY RANDOM() on large tables** — use TABLESAMPLE BERNOULLI for random sampling; it is O(n) not O(n log n).
6. **Use ROUND with negative scale** for human-friendly number formatting: `ROUND(1234567, -3)` = 1235000.
7. **Test arithmetic with edge cases**: zero, NULL, very large numbers, and negative numbers. Math functions have different NULL, zero, and negative behaviors that differ from function to function.

Performance Considerations

Arithmetic in WHERE

```
-- SLOW: function/expression on column prevents index use
WHERE  ROUND(price, 0) = 50

-- FAST: move arithmetic to the right side
WHERE  price BETWEEN 49.5 AND 50.5
```

Aggregating with NUMERIC

NUMERIC arithmetic is slower than integer or float arithmetic because it is arbitrary-precision software arithmetic. For pure performance (e.g., analytics), DOUBLE PRECISION is faster; for correctness (money), NUMERIC is required.

RANDOM() Cost

`RANDOM()` calls a pseudo-random number generator per row. For queries selecting millions of rows with a RANDOM() expression, this is significant. Pre-generate random values or use TABLESAMPLE.

Integer vs BIGINT

Integer arithmetic is slightly faster than BIGINT on 32-bit operations. Only use BIGINT when values may exceed 2.1 billion.

Interview Questions

1. What is the difference between ROUND, TRUNC, CEIL, and FLOOR for negative numbers?
2. Why does `10 / 3` return 3 in SQL, and how do you get 3.333?
3. What is the difference between `NUMERIC` and `DOUBLE PRECISION`?
4. How does GREATEST differ from the MAX aggregate function?
5. What does `GREATEST(1, NULL, 3)` return and why?
6. How would you safely perform division without risking a "division by zero" error?
7. What is the difference between `LOG(x)` and `LN(x)` in PostgreSQL?
8. How would you generate a random integer between 50 and 100?
9. When would you use TABLESAMPLE instead of ORDER BY RANDOM()?
10. How does integer division interact with type promotion in PostgreSQL?

Interview Answers

Q1: ROUND/TRUNC/CEIL/FLOOR for negatives? For -4.7: ROUND = -5 (away from zero); TRUNC = -4 (toward zero, chops decimal); CEIL = -4 (toward +infinity); FLOOR = -5 (toward -infinity). ROUND and FLOOR agree on negatives; TRUNC and CEIL agree on negatives.

Q2: `10 / 3` = 3? When both operands are integers, PostgreSQL performs integer (truncating) division. Result type is integer, fractional part discarded. To get a decimal result, cast one operand to float or numeric: `10::FLOAT / 3` or `10 / 3.0`.

Q3: NUMERIC vs DOUBLE PRECISION? NUMERIC is exact arbitrary-precision arithmetic — no floating-point rounding errors; slower; used for money/accounting. DOUBLE PRECISION is 8-byte IEEE 754 floating-point — fast but subject to rounding errors (0.1 + 0.2 ≠ 0.3 exactly). Use NUMERIC for any calculation where exactness matters.

Q4: GREATEST vs MAX? GREATEST compares values across columns within the same row: `GREATEST(col1, col2, col3)` for each row. MAX is an aggregate function that finds the maximum value of one column across multiple rows in a group.

Q5: `GREATEST(1, NULL, 3)`? Returns NULL. Unlike COALESCE, GREATEST returns NULL if any argument is NULL. To ignore NULLs, wrap arguments: `GREATEST(COALESCE(1,0), COALESCE(NULL,0), COALESCE(3,0))` = 3.

Q6: Safe division? Use `NULLIF(denominator, 0)` to convert 0 to NULL: `numerator / NULLIF(denominator, 0)`. Division by NULL returns NULL instead of raising ERROR: division by zero.

Q7: LOG vs LN? `LN(x)` = natural logarithm (base e). `LOG(x)` = log base 10. `LOG(base, x)` = log of x in the given base. This differs from some languages where `log()` is the natural log.

Q8: Random integer 50–100? `FLOOR(RANDOM() * 51 + 50)::INT` — RANDOM() gives [0, 1), multiply by range size 51 gives [0, 51), add 50 gives [50, 101), FLOOR gives {50, 51, ..., 100}.

Q9: TABLESAMPLE vs ORDER BY RANDOM()? ORDER BY RANDOM() evaluates RANDOM() for every row, then sorts all rows — O(n log n), reads the entire table. TABLESAMPLE samples at the storage page level before reading all rows — O(n) and can skip pages entirely. Use TABLESAMPLE for large table sampling; ORDER BY RANDOM() for small tables or when exact sample size matters.

Q10: Integer division and type promotion? PostgreSQL does not automatically promote integers to float. `INTEGER / INTEGER = INTEGER`. Promotion happens when at least one operand is a higher-precision type: `INTEGER / NUMERIC = NUMERIC`, `INTEGER / FLOAT = FLOAT`. To force promotion, cast explicitly.

Hands-on Exercises

Setup:

```
CREATE TABLE financial_data (
    id          SERIAL PRIMARY KEY,
    product     TEXT,
    revenue     NUMERIC(14, 6),
    cost        NUMERIC(14, 6),
    units_sold  INT,
    units_returned INT DEFAULT 0
);

INSERT INTO financial_data (product, revenue, cost, units_sold, units_returned)
VALUES
('Widget A',  15750.123456,  8250.675000, 525,  12),
('Widget B',  32000.000000, 18500.000000, 640,   0),
('Gadget C',   8950.500000,  4200.333333, 179,   5),
('Sprocket D', 5000.000000,  5000.000000, 100,   0),
('Sensor E',  42500.750000, 30000.100000, 850,  30),
('Doohickey',     0.000000,   500.000000,   0, NULL);
```

Exercise 1: Calculate gross profit (revenue - cost) and gross margin percentage, rounded to 2 decimal places. Handle the case where revenue is 0.

Exercise 2: Find the effective units sold (units_sold - COALESCE(units_returned, 0)) and calculate average revenue per effective unit, truncated to 4 decimal places.

Exercise 3: Classify each product's margin as 'High' (>40%), 'Medium' (20-40%), 'Low' (<20%), or 'Loss' (negative margin).

Exercise 4: Generate a report showing revenue rounded to the nearest 100, and cost truncated to the nearest 100.

Exercise 5: Calculate the compound growth rate assuming revenue grows at 7.5% annually. Show the projected revenue for 1, 3, and 5 years.

Solutions

```
-- Exercise 1
SELECT product,
       ROUND(revenue - cost, 2) AS gross_profit,
       ROUND((revenue - cost) / NULLIF(revenue, 0) * 100, 2) AS gross_margin_pct
FROM   financial_data;

-- Exercise 2
SELECT product,
       units_sold - COALESCE(units_returned, 0) AS net_units,
       TRUNC(revenue / NULLIF(units_sold - COALESCE(units_returned, 0), 0), 4)
           AS avg_revenue_per_unit
```

```sql
FROM   financial_data;

-- Exercise 3
SELECT product,
       ROUND((revenue - cost) / NULLIF(revenue, 0) * 100, 2) AS margin_pct,
       CASE
           WHEN revenue = 0 OR cost > revenue THEN 'Loss'
           WHEN (revenue - cost) / revenue >= 0.40 THEN 'High'
           WHEN (revenue - cost) / revenue >= 0.20 THEN 'Medium'
           ELSE 'Low'
       END AS margin_tier
FROM   financial_data;

-- Exercise 4
SELECT product,
       revenue,
       ROUND(revenue, -2) AS revenue_rounded_100,
       TRUNC(cost, -2)    AS cost_truncated_100
FROM   financial_data;

-- Exercise 5
SELECT product,
       ROUND(revenue, 2)                          AS current_revenue,
       ROUND(revenue * POWER(1.075, 1), 2)        AS year_1,
       ROUND(revenue * POWER(1.075, 3), 2)        AS year_3,
       ROUND(revenue * POWER(1.075, 5), 2)        AS year_5
FROM   financial_data
WHERE  revenue > 0;
```

Advanced Notes

`numeric_scale` and `numeric_precision` in pg_attribute

```sql
-- Inspect column precision/scale
SELECT column_name, data_type, numeric_precision, numeric_scale
FROM   information_schema.columns
WHERE  table_name = 'financial_data';
```

`width_bucket` for Histograms

```sql
-- Bucket revenue into 5 equal-width buckets between 0 and 50000
SELECT width_bucket(revenue, 0, 50000, 5) AS bucket,
       COUNT(*) AS count
FROM   financial_data
GROUP BY bucket
ORDER BY bucket;
```

Range Types for Numeric Ranges

PostgreSQL has built-in range types for numeric intervals:

```
SELECT numrange(1.5, 10.5);              -- [1.5, 10.5)
SELECT numrange(1.5, 10.5) @> 5.0;      -- TRUE: 5.0 is within range
SELECT numrange(1, 5) * numrange(3, 7); -- intersection: [3, 5)
```

`pg_numeric` and Infinity

NUMERIC can store infinity and NaN (not-a-number):

```
SELECT 'infinity'::NUMERIC;     -- infinity
SELECT 'NaN'::NUMERIC;          -- NaN
SELECT 'NaN'::NUMERIC + 5;      -- NaN (NaN propagates)
```

Cross-References

- **Previous:** 06_string_functions.md — String manipulation and type casting
- **Next:** 08_date_time_functions.md — Date/time functions and interval arithmetic
- **Related:** 05_null_handling.md — NULL in arithmetic, NULLIF for safe division
- **Related (03_Intermediate):** `03_Intermediate_SQL/02_aggregations.md` — SUM, AVG, ROUND in GROUP BY
- **Related (03_Intermediate):** `03_Intermediate_SQL/03_window_functions.md` — Running totals, percentile functions